ChatGPT said:

An **LSM Tree (Log-Structured Merge Tree)** has multiple levels to efficiently manage large volumes of writes while keeping read and storage costs manageable.

The fundamental reason is:

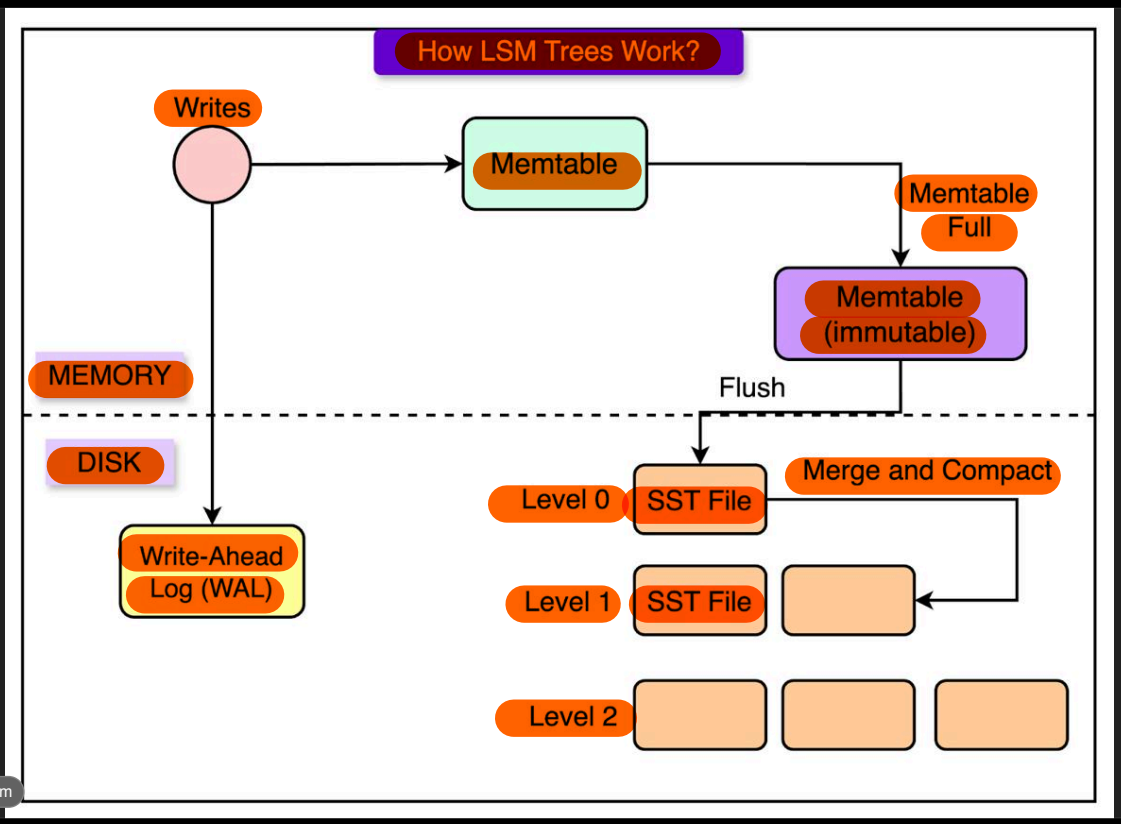
LSM Trees progressively organize and merge sorted data across multiple levels so that writes remain efficient without allowing the data structure to grow indefinitely.

Let's understand this from a database engineering and system design perspective.

1. The problem: Why not keep everything in one level?

Imagine a database receiving millions of writes per second.

An LSM Tree initially stores writes in an in-memory data structure called a **MemTable**. When the MemTable reaches a configured size, it is flushed to disk as an immutable, sorted file called an SSTable (Sorted String Table).



Suppose we continuously flush MemTables to disk:

```
Plain text
MemTable
  |
  v
SSTable-1
SSTable-2
SSTable-3
SSTable-4
SSTable-5
SSTable-6
...
SSTable-N
```

This creates several problems:

1. **Read amplification:** A database might need to search many SSTables to find a key.
2. **Space amplification:** Multiple versions of the same key may exist in different SSTables.
3. **Compaction overhead:** Too many SSTables increase the cost of maintaining the database.
4. **Tombstone accumulation:** Deleted records may remain in older SSTables.

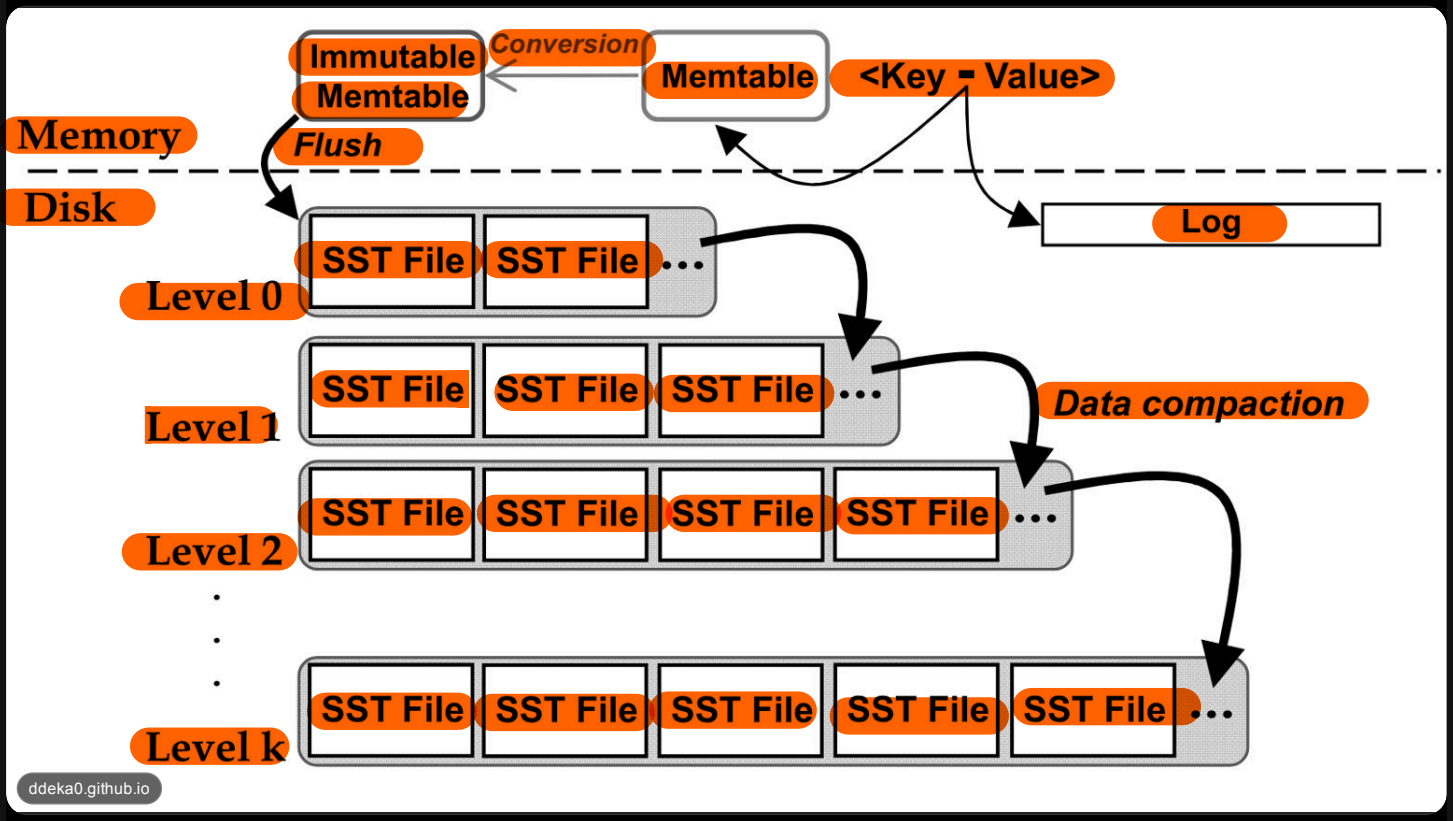
We need a mechanism to progressively consolidate these files.

That's where multiple levels come in.

2. Why multiple levels?

An LSM Tree organizes SSTables into levels.

A typical leveled-compaction architecture looks like this:



Plain text

```
            Writes
              |
              v
           MemTable
              |
              v
         Flush to disk
              |
              v
         Level 0 (L0)
        /     |     \
    SST1   SST2    SST3
              |
         Compaction
              |
              v
         Level 1 (L1)
    [SST1] [SST2] [SST3]
              |
         Compaction
              |
              v
         Level 2 (L2)
  [SST1] [SST2] [SST3] [SST4]
              |
         Compaction
              |
              v
         Level 3 (L3)
[SST1] [SST2] [SST3] [SST4] ...
```

Each level has a specific purpose:

Level	Purpose
MemTable	Absorb incoming writes in memory
L0	Receive freshly flushed SSTables
L1	Consolidate L0 data into sorted, non-overlapping SSTables
L2	Store progressively more consolidated data
L3+	Store increasingly larger amounts of historical data

The important property of leveled compaction is:

- SSTables within L0 may overlap in their key ranges.
- SSTables within L1 and subsequent levels generally have non-overlapping key ranges.
- Each subsequent level is typically larger than the previous level.

For example, with a level size multiplier of 10:

Level	Approximate capacity
L0	256 MB
L1	256 MB
L2	2.56 GB
L3	25.6 GB
L4	256 GB

These are illustrative values, not universal defaults.

3. How do multiple levels improve performance?

There are three important dimensions.

A. Reduce read amplification

Suppose you have 1 million keys distributed across 100 SSTables.

Without levels, finding a key might require checking many SSTables.

With leveled compaction:

- Each level has sorted SSTables.
- SSTables within a level have non-overlapping key ranges.
- The database can use an index or Bloom filter to identify candidate SSTables.
- It can efficiently narrow the search to a small number of files per level.

For example:

Plain text

```
Search key = user:1001

L0:
  SST1 -> Maybe
  SST2 -> Maybe
  SST3 -> Maybe

L1:
  SST4 -> No (key range mismatch)
  SST5 -> Yes

L2:
  SST6 -> No (key range mismatch)
  SST7 -> Yes

L3:
  SST8 -> Yes
```

The database searches relevant files and determines which version of the key is the newest.

Important: Multiple levels do not eliminate read amplification. They make it more predictable and manageable.

B. Control space amplification

Consider the following writes:

```
Plain text
Time 1: user:1001 = "John"
Time 2: user:1001 = "Johnny"
Time 3: user:1001 = "John Smith"
```

Different SSTables might contain:

```
Plain text
L0:
  user:1001 = "John Smith"

L1:
  user:1001 = "Johnny"

L2:
  user:1001 = "John"
```

Only the newest value is logically valid.

During compaction, the database merges sorted SSTables and discards obsolete versions when it is safe to do so.

Result:

```
Plain text
L2:
  user:1001 = "John Smith"
```

This reduces unnecessary disk consumption.

C. Control compaction costs

Suppose all data were stored in a single enormous sorted file.

Every time new data arrived, merging it with the existing data could involve substantial I/O.

Multiple levels allow incremental compaction.

Instead of repeatedly rewriting the entire dataset, the database progressively moves and merges data between levels.

This is a trade-off: compaction still generates write amplification, but the work is distributed across levels.

4. Why does each level become larger?

This is one of the most important design decisions.

Typically, an LSM Tree uses a level size multiplier, often denoted by T.

$$Size(L_{i+1}) = T \times Size(L_i)$$

For example, if $T = 10$:

```
Plain text
```

```
L0 = 256 MB
L1 = 256 MB
L2 = 2.56 GB
L3 = 25.6 GB
L4 = 256 GB
```

Why?

Because older data generally accumulates in larger quantities.

A small number of upper levels can absorb frequent incoming writes, while larger lower levels accommodate the growing dataset.

This creates a logarithmic number of levels relative to total storage capacity.

$$NumberOfLevels \approx \log_T \left(\frac{TotalData}{BaseLevelSize} \right)$$

For example, with 1 TB of data, a 256 MB base level and a multiplier of 10:

$$N \approx \log_{10}(4096) \approx 3.61$$

So approximately four growth steps beyond the base level are sufficient.

5. What happens during compaction?

Let's understand the process.

Suppose L1 contains:

Plain text

```
SSTable A: [1, 100]
SSTable B: [101, 200]
SSTable C: [201, 300]
```

New SSTables arrive from L0:

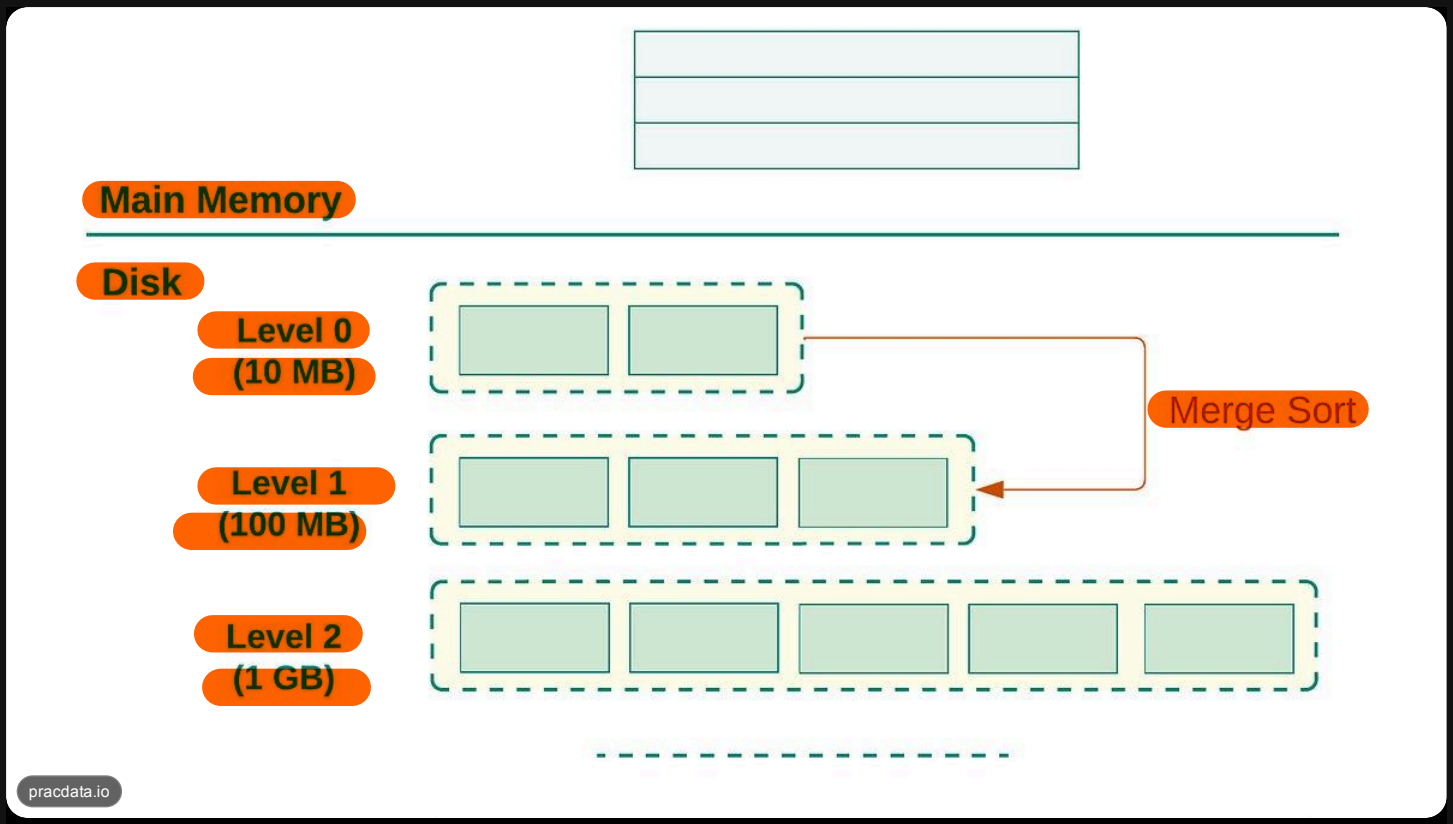
Plain text

```
SSTable D: [50, 150]
```

The key ranges overlap.

To maintain the non-overlapping property of L1, the database:

1. Selects SSTable D from L0.
2. Identifies overlapping SSTables in L1 (A and B).
3. Reads their sorted records.
4. Performs a sorted merge.
5. Resolves duplicate keys using version information.
6. Removes obsolete versions and eligible tombstones.
7. Writes new SSTables into L1.
8. Deletes the old SSTables once the replacement is safely installed.



This process is called **compaction**.

6. Why not use just two levels?

You might ask: why not have L0 and L1 only?

You can. Some LSM implementations use different organization strategies, including tiered or size-tiered approaches.

However, with only two levels:

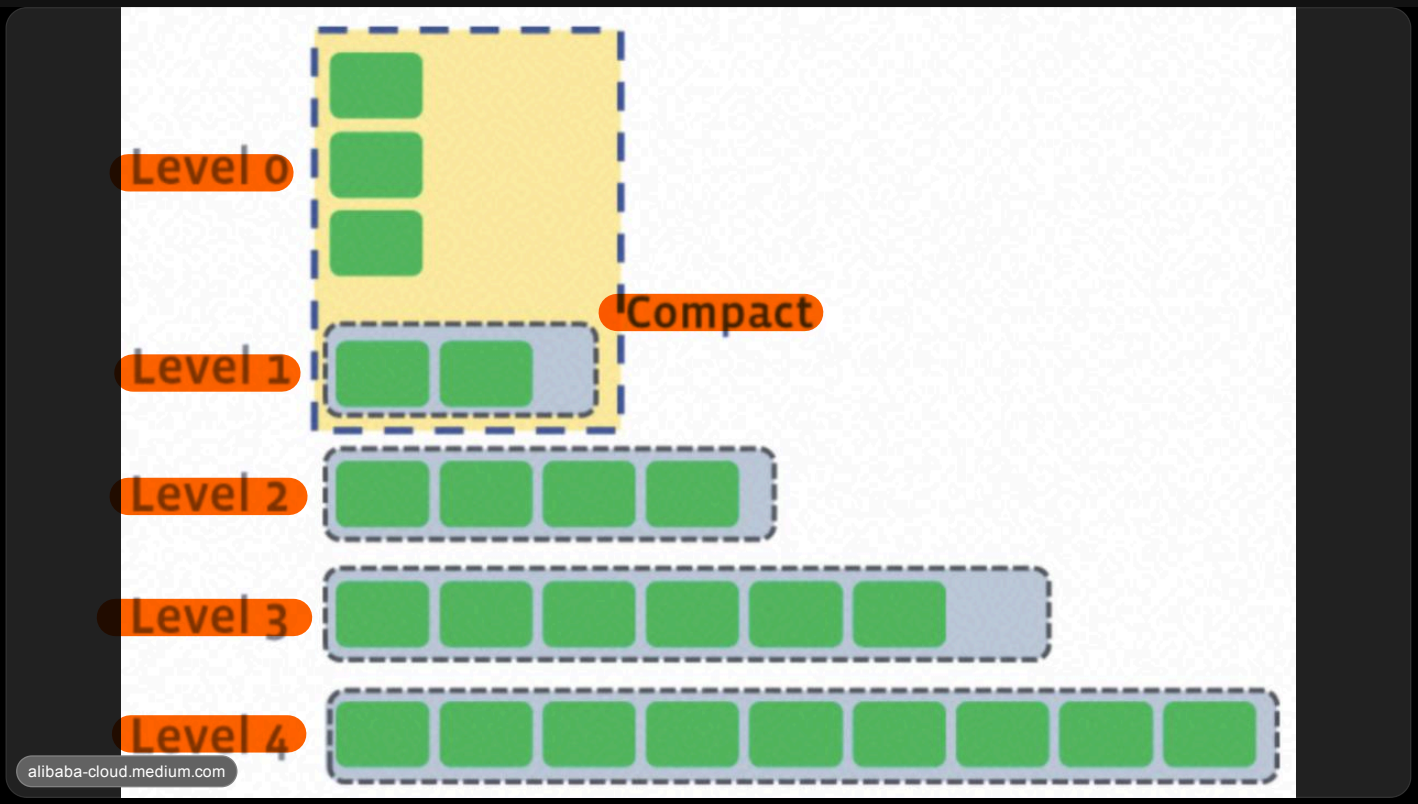
- L1 can become extremely large.
- Compaction can involve rewriting substantial amounts of data.
- Maintaining sorted, non-overlapping ranges becomes expensive as the dataset grows.

Multiple levels provide a way to distribute this work progressively.

7. Leveled vs. Tiered compaction

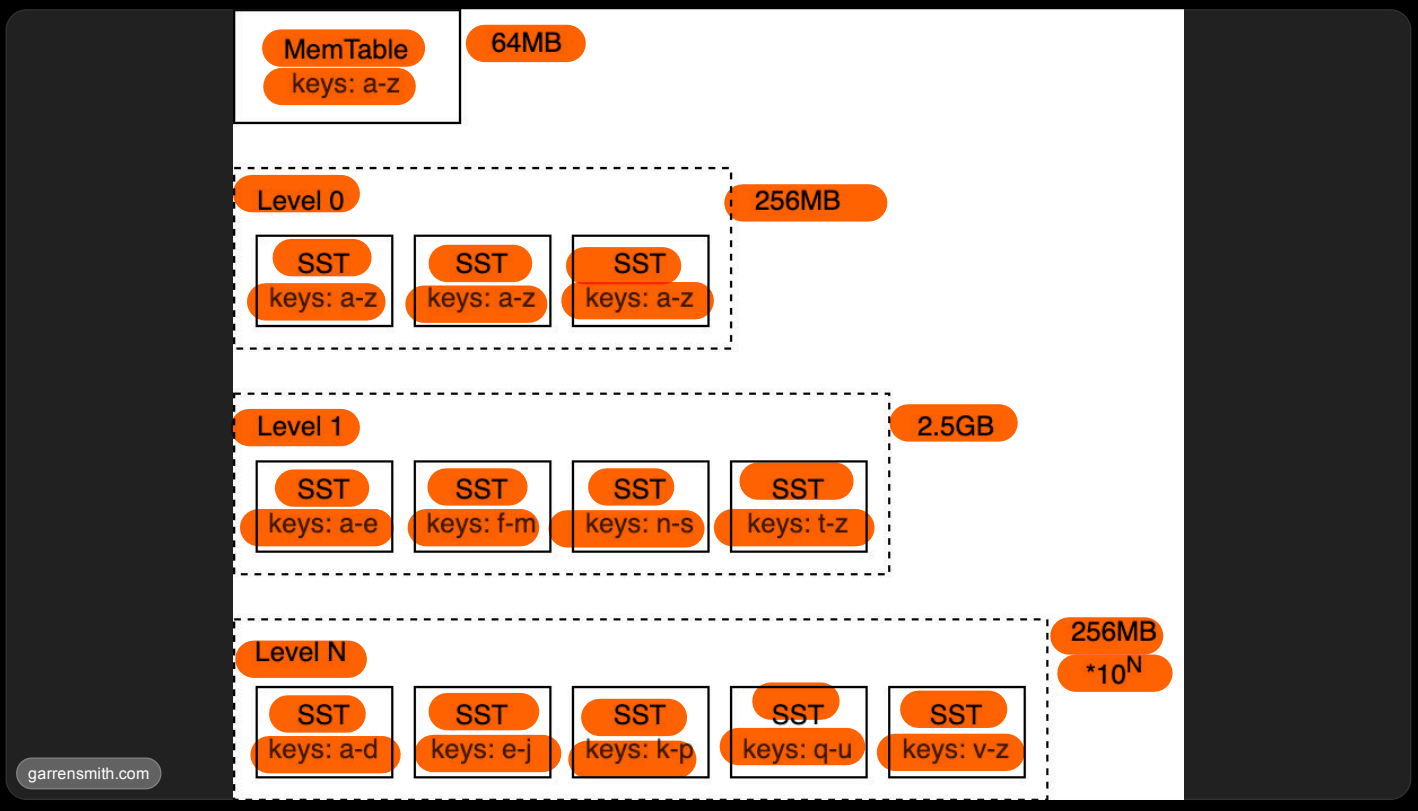
There are two important compaction strategies.

Leveled compaction



- SSTables in a level generally have non-overlapping key ranges.
- Compaction merges overlapping files into the next level.
- Lower read amplification.
- Typically higher write amplification.

Tiered (size-tiered) compaction



- Multiple sorted runs can coexist at the same tier.
- Files are merged when enough similarly sized files accumulate.
- Typically lower write amplification.
- Generally higher read and space amplification.

Characteristic	Leveled	Tiered

Read amplification	Lower	Higher
Write amplification	Higher	Lower
Space amplification	Lower	Higher
Compaction I/O	More frequent	Less frequent
Suitable for	Read-heavy workloads	Write-heavy workloads

These are general tendencies; actual behavior depends on configuration and workload.

8. Real-world database implementations

Understanding how databases use LSM Trees is particularly important for system design.

Database	Storage architecture
RocksDB	LSM Tree with configurable compaction strategies
LevelDB	LSM Tree with leveled compaction
Apache Cassandra	LSM-based storage engine with configurable compaction strategies
ScyllaDB	LSM-based storage engine
HBase	LSM-inspired StoreFile and compaction architecture

9. Staff/Principal Engineer perspective

When designing a system using an LSM-based database, you should understand these trade-offs:

Concern	Engineering consideration
Write throughput	MemTable size, WAL, flush frequency
Read latency	Number of levels, Bloom filters, block cache
Write amplification	Compaction strategy and level multiplier
Space amplification	Obsolete versions, tombstones, compaction
Disk I/O	Compaction scheduling and bandwidth
Tail latency	Background compaction competing with foreground reads/writes
Storage capacity	Temporary space needed during compaction
Operational stability	Compaction backlog and disk utilization

A particularly important operational metric is **pending compaction bytes**.

If incoming write throughput exceeds the sustainable compaction throughput, the compaction backlog grows. Eventually, the database may throttle writes or experience significant latency degradation.

---

10. The key takeaway

Think of multiple LSM levels as a hierarchical storage organization:

- **L0:** Quickly absorb newly flushed data.
- **L1:** Consolidate recently written data.
- **L2+:** Organize increasingly larger portions of the dataset.
- **Compaction:** Merge data, eliminate obsolete versions, and maintain the organization.

The fundamental objective is to balance three competing costs:

$$\boxed{Read\ Amplification \quad \leftrightarrow \quad Write\ Amplification \quad \leftrightarrow \quad Space\ Amplification}$$

There is no configuration that minimizes all three simultaneously.

Staff Engineer insight: Multiple levels are not simply a way to organize SSTables. They are a mechanism for controlling the long-term I/O economics of a database under sustained write workloads. Understanding this trade-off is essential when choosing between an LSM-based engine and a B-Tree-

based storage engine.